National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	12
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	SQL/PSM
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

SQL / PSM (SQL / Persistent Stored Modules)

SQL/PSM refers to the procedural extension of SQL that allows writing logic such as variables, conditions, and loops inside the database. In MySQL, this is implemented as **Stored Programs**, including procedures, functions, and triggers. It enables embedding business logic directly within the database instead of the application layer.

1 Triggers

A trigger is a database object that automatically executes in response to certain events on a table such as `INSERT`, `UPDATE`, or `DELETE`.

A trigger is procedural SQL code that runs automatically when certain data changes occur in a table.

1.1 Properties

- A trigger is executed automatically by the database (no manual call needed)
- It is fired **before or after** a data modification event
- A trigger is always associated with a specific table
- A table can have multiple different triggers defined on it
- Triggers are executed as part of the same transaction that caused them. If the transaction fails, the trigger's actions are also rolled back

1.2 Why and When to Use Triggers?

Triggers are used to **automate logic at the database level**.

Common use cases include:

- Enforcing business rules automatically
- Maintaining audit logs (tracking changes)
- Preventing invalid operations
- Implementing soft (lazy) deletion
- Automatically updating derived or dependent data
- Ensuring data consistency across tables

1.3 Types of Triggers

Triggers are classified based on **when they execute** relative to a data operation.

Type	Description
`BEFORE INSERT`	Executes before new data is inserted into a table
`AFTER INSERT`	Executes after data has been inserted
`BEFORE UPDATE`	Executes before existing data is updated
`AFTER UPDATE`	Executes after data has been updated
`BEFORE DELETE`	Executes before a row is deleted
`AFTER DELETE`	Executes after a row has been deleted

1.4 General Syntax

```
CREATE TRIGGER trigger_name
BEFORE/AFTER INSERT/UPDATE/DELETE ON table_name
FOR EACH ROW
BEGIN
    -- Trigger body containing SQL statements
    -- Executes once for every (inserted/updated/deleted) row
END
```

1.5 Examples

- Logs new user data into the audit table when a new record is inserted into the `users` table.

```
DELIMITER //

CREATE TRIGGER log_users_insert
AFTER INSERT ON users
FOR EACH ROW
BEGIN
    INSERT INTO users_logs (user_id, username, new_password,
action_performed)
    VALUES (NEW.id, NEW.username, NEW.password, 'INSERT');
END//

DELIMITER ;
```

- Logs both previous and updated user data whenever a record in the `users` table is modified.

```
DELIMITER //

CREATE TRIGGER log_users_update
AFTER UPDATE ON users
FOR EACH ROW
BEGIN
    INSERT INTO users_logs (user_id, username, old_password,
new_password, action_performed)
    VALUES (OLD.id, OLD.username, OLD.password, NEW.password,
'UPDATE');
END//

DELIMITER ;
```

- Logs existing user data before a record is deleted from the `users` table.

```
DELIMITER //

CREATE TRIGGER log_users_delete
BEFORE DELETE ON users
FOR EACH ROW
BEGIN
    INSERT INTO users_logs (user_id, username, old_password,
action_performed)
    VALUES (OLD.id, OLD.username, OLD.password, 'DELETE');
END//

DELIMITER ;
```

1.6 Components of a Trigger

Triggers consist of several important elements that control how and when they execute, and how they access data.

1.6.1 The DELIMITER Keyword

- By default, MySQL uses the semicolon (`;`) as a statement terminator. However, a trigger body typically contains multiple SQL statements, each ending with a semicolon.
- To prevent MySQL from prematurely ending the trigger definition, the delimiter is temporarily changed (commonly to `//` or `$$`) before creating the trigger.
- After the trigger definition is complete, the delimiter is restored back to `;`.
- This allows MySQL to treat the entire trigger body as a single statement.

1.6.2 The NEW Keyword

The `NEW` keyword represents the **new version of a row** that is being inserted or updated.
It is used in:

- `INSERT` triggers: refers to the row being inserted
- `UPDATE` triggers: refers to the updated (new) values

In `BEFORE` triggers, the `NEW` values can be modified before they are written to the table. This is useful for enforcing rules or automatically adjusting values.

In AFTER **triggers**, the row has already been processed, so NEW values are **read-only** and cannot be changed.

1.6.3 The OLD Keyword

The OLD keyword represents the **previous version of a row before it is modified or deleted**.
It is used in:

- UPDATE trigger: refers to the values before the update
- DELETE trigger: refers to the row being deleted

The OLD values are **read-only** and cannot be modified.

2 Stored Procedures

A stored procedure is a precompiled set of SQL statements stored in the database and executed as a single unit. It **supports procedural logic** such as variables, loops, and conditional statements.

Advantages:

- Stored procedures encapsulate business logic inside the database, allowing complex operations to be handled centrally instead of being written in application code.
- They promote code reuse by allowing the same SQL logic to be executed multiple times without rewriting, improving maintainability and reducing redundancy.
- They improve performance because they are precompiled and executed on the database server, reducing query execution overhead.
- They reduce network traffic by executing multiple SQL operations in a single database call instead of multiple requests.
- They also support procedural programming features like variables, conditionals and loops, enabling complex logic such as validation, calculations, and decision-making inside SQL.

General Syntax:

Procedures are defined using the CREATE PROCEDURE command and executed using the CALL command.

```
-- Defining the procedure
CREATE PROCEDURE procedure_name(IN/OUT/INOUT parameter datatype, ...)
BEGIN
```

```
    -- SQL statements
END;

-- Calling the procedure
CALL procedure_name(arguments);
```

Example:

```
DELIMITER //

CREATE PROCEDURE show_deleted_users()
BEGIN
    SELECT * FROM users_logs WHERE action_performed = 'DELETE';
END//

DELIMITER ;

CALL show_deleted_users();
```

2.1 The IN, OUT, INOUT Parameters

2.1.1 IN Parameter

Used to pass values **into** the procedure. It cannot be modified inside the procedure.

Example:

```
DELIMITER //

CREATE PROCEDURE get_user(IN uid INT)
BEGIN
    SELECT * FROM users WHERE id = uid;
END//

DELIMITER ;

CALL get_user(1);
```

2.1.2 OUT Parameter

Used to return values **from** the procedure to the caller.

Example:

```
DELIMITER //

CREATE PROCEDURE count_users(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM users;
END//

DELIMITER ;

CALL count_users(@result); -- passing session local variable
SELECT @result;
```

2.1.3 INOUT Parameter

Used to both pass a value **in and return** a modified value back.

Example:

```
DELIMITER //

CREATE PROCEDURE increase_value(INOUT num INT)
BEGIN
    SET num = num + 10;
END//


DELIMITER ;

SET @val = 5; -- defining session local variable
CALL increase_value(@val);
SELECT @val;
```

2.2 Variables, Conditionals, Loops

General Syntax:

```
DELIMITER //

CREATE PROCEDURE procedure_name(IN/OUT/INOUT name datatype, ...)
BEGIN
    -- Variable declarations
    DECLARE var_name datatype DEFAULT value;

    -- Variable assignment
    SET var_name = value;

    -- Variable declarations from a query
    SELECT column INTO var_name FROM table_name WHERE ...;

    -- Control flow
    IF condition THEN
        -- statements
    ELSEIF condition THEN
        -- statements
    ELSE
        -- statements
    END IF;

    -- Loops
    WHILE condition DO
        -- statements
    END WHILE;

    -- SQL statements
END//

DELIMITER ;
```

Examples:

- Insert multiple rows using loop:

```
CREATE PROCEDURE insert_rows(IN total_rows INT)

BEGIN
```

```
    DECLARE i INT DEFAULT 0;

    WHILE i < total_rows DO

        INSERT INTO sample_table (name, age, salary)
        VALUES (
            CONCAT('Name_', i),
            FLOOR(RAND() * 60) + 18,
            (RAND() * 1000) + 3000
        );

        SET i = i + 1;

    END WHILE;

END$$

DELIMITER ;

CALL insert_random_data(100);
```

- …

```
DELIMITER //

CREATE PROCEDURE give_bonus(IN emp_id INT)
BEGIN
    DECLARE sal DECIMAL(8,2);

    SELECT salary INTO sal FROM employees WHERE id = emp_id;

    IF sal < 5000 THEN
        UPDATE employees SET salary = salary + 500 WHERE id =
emp_id;
    ELSE
        UPDATE employees SET salary = salary + 200 WHERE id =
emp_id;
    END IF;
END//
```

```
DELIMITER ;

CALL give_bonus(3);
```

3 User Defined Functions (UDFs)

A function is similar to a stored procedure but must return exactly one value. It can be used directly inside SQL statements such as `SELECT`, `WHERE`, or `ORDER BY`.

Functions are mainly used for reusable computations that can be embedded inside queries.

3.1 Procedure vs Function

Feature	**Procedure**	**Function**
Return Value	Optional	Mandatory
Usage	CALL using `CALL`	Used inside SQL queries
Purpose	Performs actions / operations	Returns computed result
Output	Multiple results possible	Only one value

3.2 General Syntax

Functions are created using `CREATE FUNCTION` and must always return a value.

- `DETERMINISTIC` means the function always produces the same output for the same input.
- `NOT DETERMINISTIC` means output may vary (e.g., random values or time-based logic).

```
DELIMITER //

CREATE FUNCTION function_name(parameter datatype, ...)
RETURNS return_datatype
DETERMINISTIC | NOT DETERMINISTIC
BEGIN
    -- SQL Statements
```

```
    -- Must return exactly one value
    RETURN value;
END//

DELIMITER ;
```

3.3 Examples

- Salary Category Function

```
DELIMITER //

CREATE FUNCTION salary_category(salary INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    IF salary >= 50000 THEN
        RETURN 'HIGH';
    ELSEIF salary >= 20000 THEN
        RETURN 'MEDIUM';
    ELSE
        RETURN 'LOW';
    END IF;
END//

DELIMITER ;

SELECT salary_category(30000);
```

- Calculate Age from Birth Year

```
DELIMITER //

CREATE FUNCTION calculate_age(birth_year INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN YEAR(CURDATE()) - birth_year;
END//
```

```
DELIMITER ;

SELECT calculate_age(2000);
```

DDL Command Variations

Command	Objects
CREATE	DATABASE, TABLE, VIEW, INDEX, TRIGGER, PROCEDURE, FUNCTION
ALTER	same as CREATE
DROP	same as CREATE
TRUNCATE	TABLE only